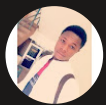


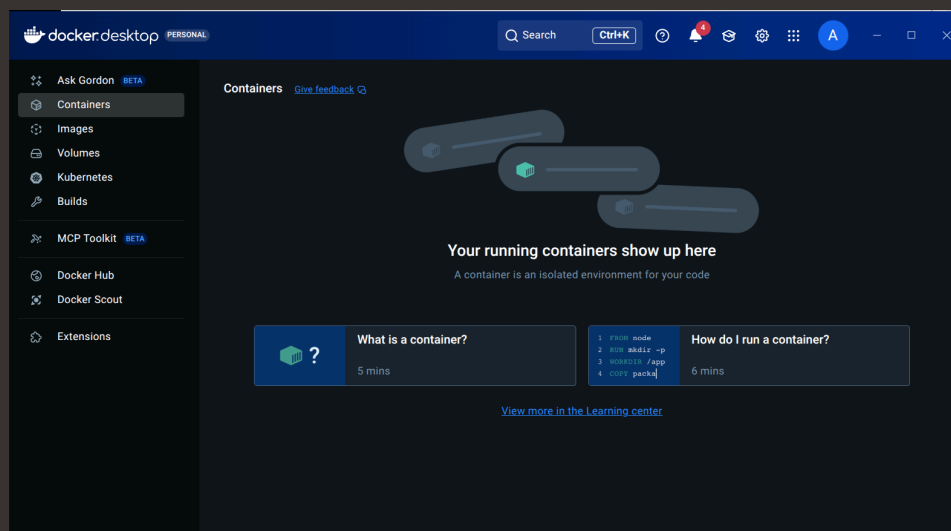


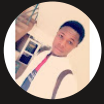
nextwork.org

Deploy an App with Docker



Ahmed Tetteh





Introducing Today's Project!

What is Docker?

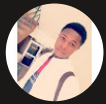
Docker is container management tool for creating and running containers, and it is useful because it ensures a reliable way to run applications or processes across different systems and environments, regardless of their underlying OS. I used Docker in this project to create a docker image and run a container using that image as it's template. This local test proved that my application could now be deployed using AWS Elastic Beanstalk.

One thing I didn't expect...

One thing I didn't expect in this project was how easy and fast it was to set up an application deployment in the cloud using AWS Elastic Beanstalk.

This project took me...

This project took me approximately 3 hours. The most challenging part was knowing the effects of the different configurations in the AWS Elastic Beanstalk. It was most rewarding to see my application live on the cloud via Elastic Beanstalk's generated public IP for app.



Understanding Containers and Docker

Containers

Containers are running instances of an image, containing all the necessary application code, dependencies and libraries all bundled together to work on any environment. They are useful because they make collaboration between developers much easier, by ensuring a stable and reliable environment for them to create and test applications faster without setting up or installing their own resources and libraries on their computer.

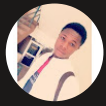
A container image is a package of optimized layers containing all the essential data needed to create a container. So, it can be thought of as the template for creating containers.

Docker

Docker is the main tool that creates and runs containers, while Docker Desktop is the user friendly program that uses Docker and runs locally on our machines.

placeholder

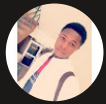
The Docker daemon is the engine or chef who does all the cooking in the background. When docker commands are run or clicked on the docker-desktop (the docker client), it takes all these commands and does the heavy lifting.



Ahmed Tetteh
NextWork Student

nextwork.org

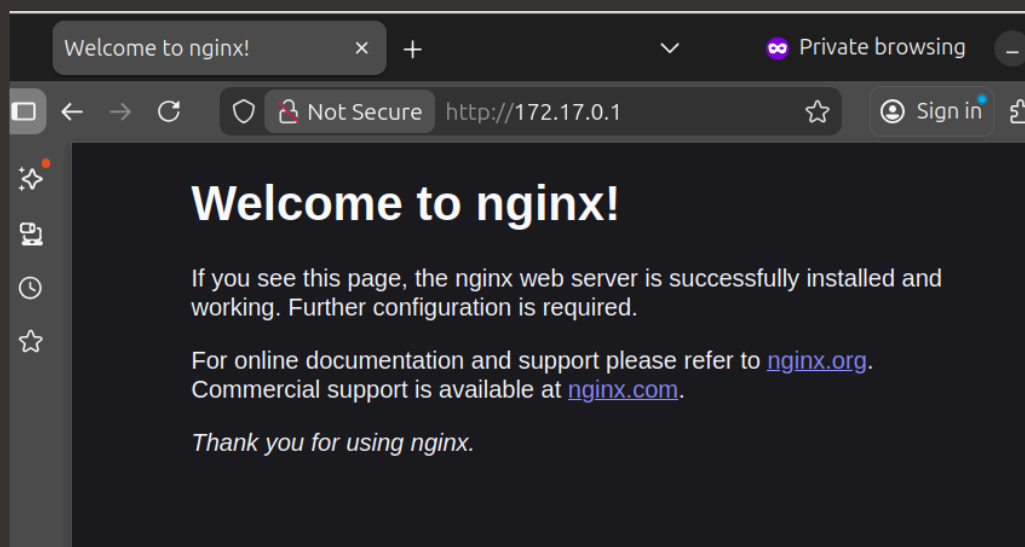
It creates docker images and runs or distributes containers.

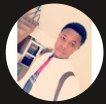


Running an Nginx Image

Nginx is a web server, that is, a program for serving web content.

The command I ran to start a new container was "docker run ". I also added some few flags by making the container run in the background using the detached mode flag (-d), and also mapped the container port to my local host port through port mapping (-p 80:80).





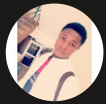
Creating a Custom Image

The Dockerfile is document containing all the instructions to build a docker image. The docker daemon then takes the Dockerfile and follows these set of instructions to build the image.

My Dockerfile tells Docker three things: 1. To use the latest version of Nginx as its base image. 2. Replaces the Nginx index.html file with our own custom index.html file in it's directory structure. 3. And exposes port 80 on the container to receive traffic, making it easier to access the web app.

The command I used to build a custom image with my Dockerfile was "docker build -t my-web-app .". The '.' at the end of the command means that docker should look for the dockerfile and other required files from the current directory.

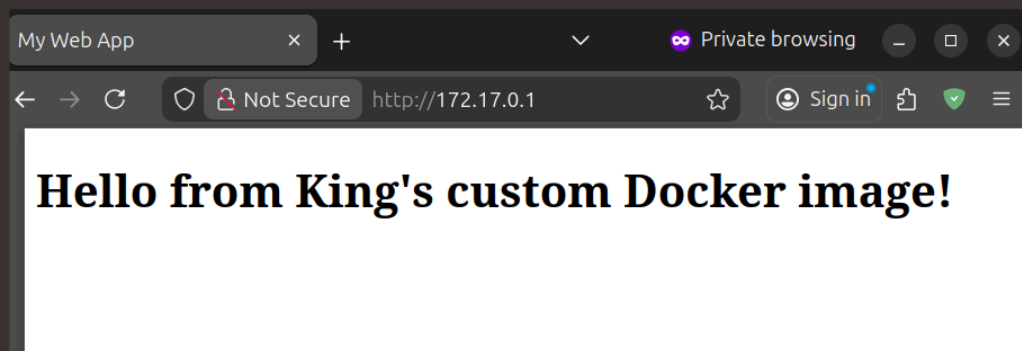
```
Terminal
GNU nano 7.2 Dockerfile *
FROM nginx:latest
COPY index.html /usr/share/nginx/html/
EXPOSE 80
█
```

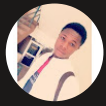


Running My Custom Image

There was an error when I ran my custom image because a previous test container I created was connected to my local host via that port. I resolved this by stopping that container using the command "docker stop CONTAINER-ID"

In this example, the container image is the custom docker image I created called my-web-app (nginx template for creating the container), while the container is simply a running instance of that image, with it's given container-id. The container serves web content based on the custom index.html file within it.

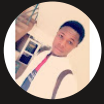




Elastic Beanstalk

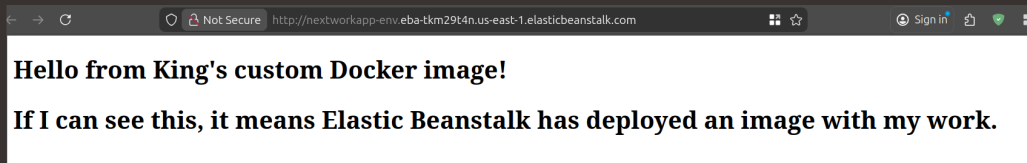
Elastic Beanstalk is a service on AWS that makes it easy to deploy and manage applications without worrying about the underlying infrastructure. It handles the load balancing, auto-scaling and more for you, so you just focus on improving your application. So, in the development phase, a developer can create and test their application locally, wrap the files in a zipped format and then export it to Elastic Beanstalk to build your docker image and deploy your application in the cloud.

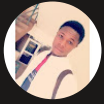
Deploying my custom image with Elastic Beanstalk took me just about a 10 minutes to set up the entire environment for my application, including the launch time.



Ahmed Tetteh
NextWork Student

nextwork.org

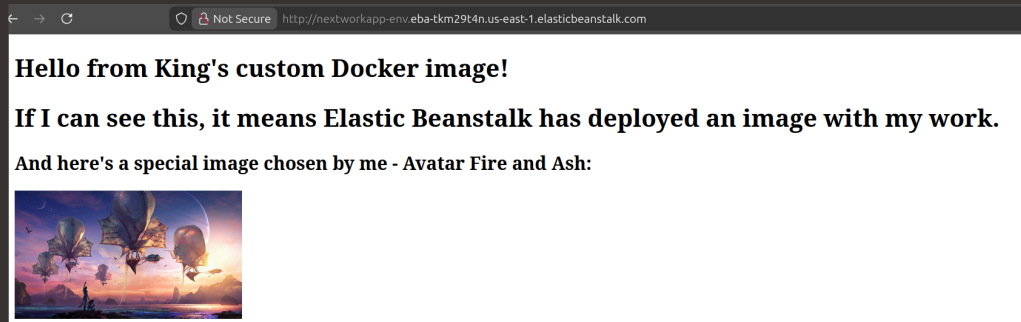
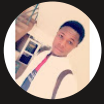




Deploying App Updates

To learn how to deploy app updates with Elastic Beanstalk, I updated my app by adding an image to the index.html file. I verified those changes by opening the file locally on my browser and the VS Code live preview extension.

My app updates didn't show up in my live environment immediately because I was running them locally. To deploy my changes, I only had to create a new zip file of Dockerfile and the updated index.html file, deploy the file to Elastic Beanstalk using the "Upload and deploy" button in my environment window. There is no need to set up a new environment since these changes are related to the application code itself, not the Elastic Beanstalk environment.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

